



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Departament d'Enginyeria Minera, Industrial
i TIC

PRÀCTICA 7

Semàfor

Jordi Garcia

Mateu Magem



1. INTRODUCCIÓ

L'objectiu d'aquesta pràctica és fer la descripció de hardware d'un dispositiu que controli un semàfor per a vehicles i vianants.

El dispositiu controlarà els tres discs del semàfor de circulació de vehicles: verd (dg), ambre (da) i vermell (dr). Així com els indicadors per als vianants: verd (wg) i vermell (wr). A més, tindrà un indicador numèric que informará del temps que els queda als vianants per a creuar el carrer.

Disposarà d'un pulsador de petició de pas per als vianants (p), que al ser activat s'iniciarà una seqüència temporitzada.

2. PROJECTES I ARQUITECTURES

2.1. Divisor de rellotge

El nostre semàfor anirà controlat amb un rellotge que comptarà en segons. Per tant, primer de tot crearem un projecte amb un divisor de clock que ens ho faci possible.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity clk_div_1s is
    generic (eoc: integer:= 100000000);
    Port ( clk : in STD_LOGIC;
          reset : in STD_LOGIC;
          clk_div_sec : out STD_LOGIC);
end clk_div_1s;

architecture Behavioral of clk_div_1s is
    signal div_cnt: integer range 0 to 100000000;
begin
    -- Divisor del clk (de 100Mhz a freqüència escollida)
    divisor_clk: process (clk, reset) begin
        if clk'event and clk = '1' then
            clk_div_sec <= '0';
            if reset = '1' then
                div_cnt <= 0;
            elsif div_cnt = eoc then
                div_cnt <= 0;
                clk_div_sec <= '1';
            else div_cnt <= div_cnt + 1;
            end if;
        end if;
    end process;
end architecture;
```

Com que el rellotge intern de la Basys 3 és de 100 MHz, dividirem la freqüència entre 100 M per tal d'aconseguir la freqüència desitjada (1Hz).

Un cop tenim el divisor de clock, creem la màquina d'estats per tal de sincronitzar les diferents etapes amb les sortides corresponents.

2.2. Màquina d'estats

La nostra màquina d'estats està composta per la següent entitat:

```
entity semafor is
  Port ( clk : in STD_LOGIC;
        p : in STD_LOGIC;
        reset : in STD_LOGIC;
        div_clk : in STD_LOGIC;
        enable_out : out STD_LOGIC;
        enable_out_b : out STD_LOGIC;
        conta : out STD_LOGIC_VECTOR (7 downto 0);
        vermell_c : out STD_LOGIC;
        ambar_c : out STD_LOGIC;
        verd_c : out STD_LOGIC;
        verd_v : out STD_LOGIC;
        vermell_v : out STD_LOGIC);
end semafor;
```

Com a entrada a destacar tenim l'entrada p, que correspon al botó per activar el pas dels vianants i el div_clk que correspon a la senyal de 1 segon que prové del divisor de rellotge. Els dos enable corresponen a l'activació del compte enrere en els displays i al pampallugueix quan quedin 5 segons.

El conta serà el valor de sortida que mostrarem en els displays quan quedin 15 segons. La resta de senyals corresponen a sortides d'un sol bit que seran per fer la codificació del semàfor en els displays restants de la placa.

```
architecture Behavioral of semafor is

  type state_sem is (no_walk, walk_req, all_stop_1, walk, end_walk, all_stop_2);
  signal temps : integer range 0 to 25;
  signal next_state, act_state: state_sem;
```

Com a senyals internes tenim la llista d'estats del nostre sistema, un valor integer que va del 0 al 25 i les senyals de sincronització d'estats.

El valor numèric ens servirà per crear el comptador per passar d'etapa en un temps determinat.



```
-- COMPTADOR:
process(div_clk) begin

    if act_state=no_walk then
        temps <= 25;
    else
        if rising_edge(div_clk) then
            if temps = 0 then
                temps <= temps - 1;
            end if;
        end if;
    end if;

end process;
```

Com ens diu l'enunciat, el temps que trigarà el semàfor en fer tot el procés des de que s'activa (premen el botó) és de 25 segons.

```
--MÀQUINA D'ESTATS:

process( reset, clk) begin
if rising_edge(clk) then
    if reset ='1' then
        act_state <= no_walk;
    else
        act_state <= next_state;
    end if;
end if;
end process;
```

El primer procés de la màquina d'estats serà el que actualitza l'estat actual en el que es troba el sistema. També programarem el reset, el qual ens posarà el sistema a l'estat inicial si l'activem.



```
process (act_state, temps, next_state, p, reset) begin

case act_state is
when no_walk =>
    if p = '1' then
        next_state <= walk_req;
    elsif reset = '1' then
        next_state <= no_walk;
    end if;

when walk_req =>
    if temps= 21 then
        next_state <= all_stop_1;
    elsif reset = '1' then
        next_state <= no_walk;
    end if;

when all_stop_1 =>
    if temps = 18 then
        next_state <= walk;
    elsif reset = '1' then
        next_state <= no_walk;
    end if;

when walk =>
    if temps = 8 then
        next_state <= end_walk;
    elsif reset = '1' then
        next_state <= no_walk;
    end if;

when end_walk =>
    if temps= 3 then
        next_state <= all_stop_2;
    elsif reset = '1' then
        next_state <= no_walk;
    end if;

when all_stop_2 =>
    if temps = 0 then
        next_state <= no_walk;
    elsif reset = '1' then
        next_state <= no_walk;
    end if;
end case;
end process;
```

El següent procés estarà compost per una sentència case. Aquest procés ens determinarà l'estat següent del nostre sistema. Aquest vindrà donat per l'estat actual en que ens trobem i el valor del temporitzador esmentat anteriorment.



```
process (act_state) begin
    if act_state = no_walk then
        verd_c <= '1';
        vermell_v <= '1';
        ambar_c <= '0';
        vermell_c <= '0';
        enable_out <= '0';
        enable_out_b <='0';

    elsif act_state = walk_req then
        verd_v <= '0';
        verd_c <= '0';
        vermell_v <= '1';
        ambar_c <= '1';
        vermell_c <= '0';
        enable_out <= '0';
        enable_out_b <='0';
    elsif act_state = all_stop_1 then
        verd_v <= '0';
        verd_c <= '0';
        vermell_v <= '1';
        ambar_c <= '0';
        vermell_c <= '1';
        enable_out <= '0';
        enable_out_b <='0';
    elsif act_state = walk then
        verd_v <= '1';
        verd_c <= '0';
        vermell_v <= '1';
        ambar_c <= '0';
        vermell_c <= '0';
        enable_out <= '1';
        enable_out_b <='0';

    elsif act_state = end_walk then
        verd_v <= '1';
        verd_c <= '0';
        vermell_v <= '1';
        ambar_c <= '0';
        vermell_c <= '0';
        enable_out <= '1';
        enable_out_b <='1';

    elsif act_state = all_stop_2 then
        verd_v <= '0';
        verd_c <= '0';
        vermell_v <= '1';
        ambar_c <= '0';
        vermell_c <= '1';
        enable_out <= '0';
        enable_out_b <='0';
    end if;
end process;

conta <= std_logic_vector(to_unsigned(temps-3,8));

end Behavioral;
```

Per últim, tindrem el procés decodificador de les sortides. Aquest activarà o no les sortides de la màquina d'estats en funció de l'estat en que ens trobem.

2.3. Visualitzador o descodificador dels displays

El visualitzador és el subprograma encarregat de representar l'estat del semàfor en els visualitzadors de 7 segments de la placa Basys 3 en funció de les ordres rebudes de la màquina d'estats.

Primerament declarem la llibreria fonamental a utilitzar, *IEEE*, i indiquem els components de la mateixa a utilitzar.

Posteriorment declarem l'entitat *traffic_light* amb les variables globals així com els ports d'entrada i sortida del subprograma.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4  use IEEE.NUMERIC_STD.ALL;
5
6  entity traffic_light is
7      generic (n: positive:=8);
8      Port ( counter : in STD_LOGIC_VECTOR (n-1 downto 0);
9            div_clk   : in STD_LOGIC;
10           div_clk1  : in STD_LOGIC;
11           clk       : in STD_LOGIC;
12           enable    : in STD_LOGIC; -- Activa el temporitzador dels vianants
13           enable_b  : in STD_LOGIC; -- Activa el parpadeig del senyal de vianants
14           red_c     : in STD_LOGIC;
15           ambar_c   : in STD_LOGIC;
16           green_c   : in STD_LOGIC;
17           red_v     : in STD_LOGIC;
18           green_v   : in STD_LOGIC;
19           reset     : in STD_LOGIC;
20           cat       : out STD_LOGIC_VECTOR (7 downto 0);
21           an        : out STD_LOGIC_VECTOR (3 downto 0));
22  end traffic_light;
```

Els ports utilitzats són:

clk	Rellotge
reset	Reset asíncron
div_clk	Divisor de rellotge per refrescar els visualitzadors de 7 segments
div_clk1	Divisor de rellotge de 1 segon
counter	Variable de compte enrere del semàfor
enable	Activa el temporitzador dels vianants
enable_b	Activa el parpadeig del temporitzador dels vianants



red_c	És 1 si el semàfor dels cotxes està vermell
ambar_c	És 1 si el semàfor dels cotxes està ambre
green_c	És 1 si el semàfor dels cotxes està verd
red_v	És 1 si el semàfor dels vianants està vermell
green_v	És 1 si el semàfor dels vianants està verd
cat	El càtode controla els segments actius dels visualitzadors de 7 segments
an	L'ànode que controla quin dels 4 visualitzadors està actiu

Segonament, definim l'arquitectura *Behavioral* del programa amb les respectives senyals.

```
24 architecture Behavioral of trafic_light is
25     signal digit_act: integer range 0 to 3;
26     signal sg, char0, char1, char2, char3: std_logic_vector(3 downto 0);
27     signal bcdcount: std_logic_vector(7 downto 0);
```

digit_act	Enter de 0 a 3 indicant quin dels 4 visualitzadors està actiu
sg	Vector de 4 bits que emmagatzema el valor a representar en el visualitzador actiu
char0 char1 char2 char3	Vectors de 4 bits que emmagatzemen els valors a representar en cadascun dels visualitzadors
bcdcount	Vector de 8 bits que emmagatzema el valor del temporitzador dels vianants convertit a BCD

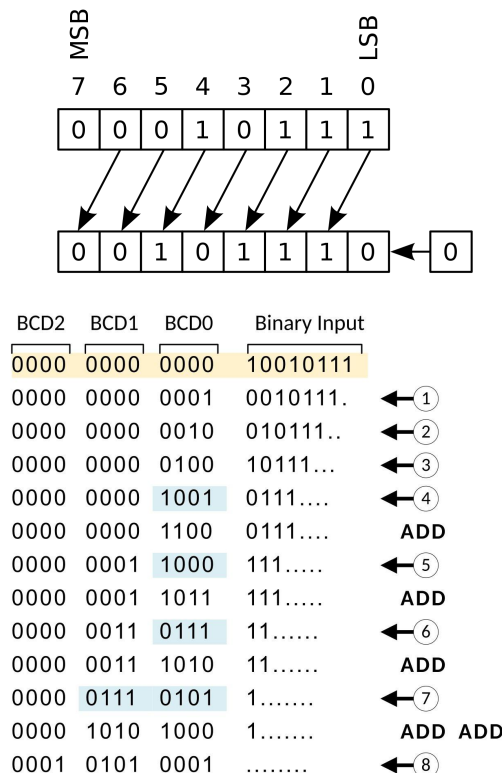
A continuació implementem una funció que converteix un vector de bits codificats en binari a un vector de bits codificat en Binary Coded Decimal (BCD) per tal de convertir el temporitzador binari facilitat per la màquina d'estats a BCD representable en els visualitzadors de 7 segments.

```

29  function to_bcd (bin : std_logic_vector(n-1 downto 0) ) return std_logic_vector is
30      variable i : integer:=0;
31      variable bcd : std_logic_vector(n-1 downto 0) := (others => '0');
32      variable bint : std_logic_vector(n-1 downto 0) := bin;
33  begin
34      for i in 0 to n-1 loop -- repeating n-1 times.
35          bcd(n-1 downto 1) := bcd(n-2 downto 0); --shifting the bits.
36          bcd(0) := bint(n-1);
37          bint(n-1 downto 1) := bint(n-2 downto 0);
38          bint(0) := '0';
39          if(i < n-1 and bcd(3 downto 0) > "0100") then --add 3 if BCD digit is greater than 4.
40              bcd(3 downto 0) := bcd(3 downto 0) + "0011";
41          end if;
42          if(i < n-1 and bcd(7 downto 4) > "0100") then --add 3 if BCD digit is greater than 4.
43              bcd(7 downto 4) := bcd(7 downto 4) + "0011";
44          end if;
45      end loop;
46      return bcd;
47  end to_bcd;
48

```

La conversió s'implementa amb l'algorisme *Double Dabble*. Es realitza desplaçant el vector binari dins el vector en BCD i sumant 0100 quan el valor d'algun dels *nibbles* (conjunts de 4 bits) és major o igual que 0101 (5 en binari) fins a desplaçar el bit binari de menor pes.



Iniciem l'arquitectura.

Utilitzem el procés *digit_actual* per actualitzar la variable *digit_act* cada pols del divisor de rellotge *div_clk* que indica quin visualitzador està actiu en cada instant.

```
49 | begin
50 | -- 2 Comptador que assigna la entrada que toca
51 | digit_actual: process(clk, reset) begin
52 |     if clk'event and clk = '1' then
53 |         if reset = '1' then
54 |             digit_act <= 0;
55 |         elsif div_clk = '1' then
56 |             if digit_act = 3 then
57 |                 digit_act <= 0;
58 |             else digit_act <= digit_act + 1;
59 |             end if;
60 |         end if;
61 |     end if;
62 | end process;
```

Utilitzem un altre procés per controlar la lògica d'il·luminació del semàfor que s'actualitza quan la màquina d'estat canvia l'estat d'algun dels semàfors. Mitjançant dues sentències condicionals es controla el semàfor dels vehicles i el dels vianants assignant al visualitzador corresponent el codi per descodificar el càtode en cadascun dels estats del semàfor (vermell, ambre i verd).

```
64 | -- Lògica d'il·luminació del semàfor
65 | illumination: process(green_c, ambar_c, red_c, green_v, red_v) begin
66 |     -- Cotxes
67 |     if green_c = '1' then
68 |         char3 <= "1100";
69 |     elsif ambar_c = '1' then
70 |         char3 <= "1011";
71 |     else --elsif red_c = '1' then
72 |         char3 <= "1010";
73 |     end if;
74 |     -- Vianants
75 |     if green_v = '1' then
76 |         char2 <= "1100";
77 |     else --elsif red_v = '1' then
78 |         char2 <= "1010";
79 |     end if;
80 | end process;
```

Un altre procés s'ocupa de lògica del temporitzador governada per la màquina d'estats. Si *enable* és 0, el temporitzador es troba desactivat. Si *enable* és 1, es mostra el temporitzador convertit a BCD als visualitzadors corresponents (els dos de la dreta). Quan *enable* i *enable_b* són 1, el temporitzador s'activa i es desactiva cada mig segon (500 ms) mitjançant la sentència *wait for* per produir un efecte intermitent.

```

82 | -- Temporitzador
83 | timer: process begin
84 |     wait on enable, enable_b;
85 |     if enable = '1' then -- Temporitzador comptant intermitent
86 |         if enable_b = '1' then
87 |             bcdcount <= to_bcd(counter);
88 |             char0 <= bcdcount(3 downto 0);
89 |             char1 <= bcdcount(7 downto 4);
90 |             wait for 500 ms;
91 |             char0 <= "1101";
92 |             char1 <= "1101";
93 |             wait for 500 ms;
94 |         else -- Temporitzador comptant de forma continua
95 |             bcdcount <= to_bcd(counter);
96 |             char0 <= bcdcount(3 downto 0);
97 |             char1 <= bcdcount(7 downto 4);
98 |         end if;
99 |     else -- Temporitzador apagat
100 |         char0 <= "1101";
101 |         char1 <= "1101";
102 |     end if;
103 | end process;

```

Dissenyem un procés *selector* que assigna el valor corresponent de cada visualitzador de 7 segments a la variable *sg* que codifica el càtode. La variable *digit_act* serveix per seleccionar quin valor s'ha de donar al càtode cada vegada que els visualitzadors s'actualitzen a ritme de *div_clk*.

```

105 | -- Valors de cadascun dels displays que s'alternen segons el div_clk (digit_act)
106 | selector: process(clk, digit_act, sg, char0, char1, char2, char3) begin
107 |     -- Assignar conjunt de switches a sg
108 |     case digit_act is
109 |         when 0 =>
110 |             sg <= char0;
111 |         when 1 =>
112 |             sg <= char1;
113 |         when 2 =>
114 |             sg <= char2;
115 |         when 3 =>
116 |             sg <= char3;
117 |     end case;
118 | end process;

```



El descodificador del càtode selecciona el valor corresponent del càtode per cada valor d'entrada com són els nombres 0-9 pel temporitzador i els estats dels semàfors vermell, ambre, verd i apagat.

```
120 | -- Selector de càtode segons sg
121 | with sg select cat <=
122 | -- Codificació dels nombres
123 |         "00000011" when "0000", --0
124 |         "10011111" when "0001", --1
125 |         "00100101" when "0010", --2
126 |         "00001101" when "0011", --3
127 |         "10011001" when "0100", --4
128 |         "01001001" when "0101", --5
129 |         "01000001" when "0110", --6
130 |         "00011111" when "0111", --7
131 |         "00000001" when "1000", --8
132 |         "00011001" when "1001", --9
133 | -- Codificació del semàfor
134 |         "00111001" when "1010", -- Semàfor vermell
135 |         "00000011" when "1011", -- Semàfor àmbar
136 |         "11000101" when "1100", -- Semàfor verd
137 |         "11111111" when "1101", -- Semàfor apagat
138 | -- Codificació de signes auxiliars
139 |         "11111101" when "1111", -- -
140 |         "11111111" when others;
```

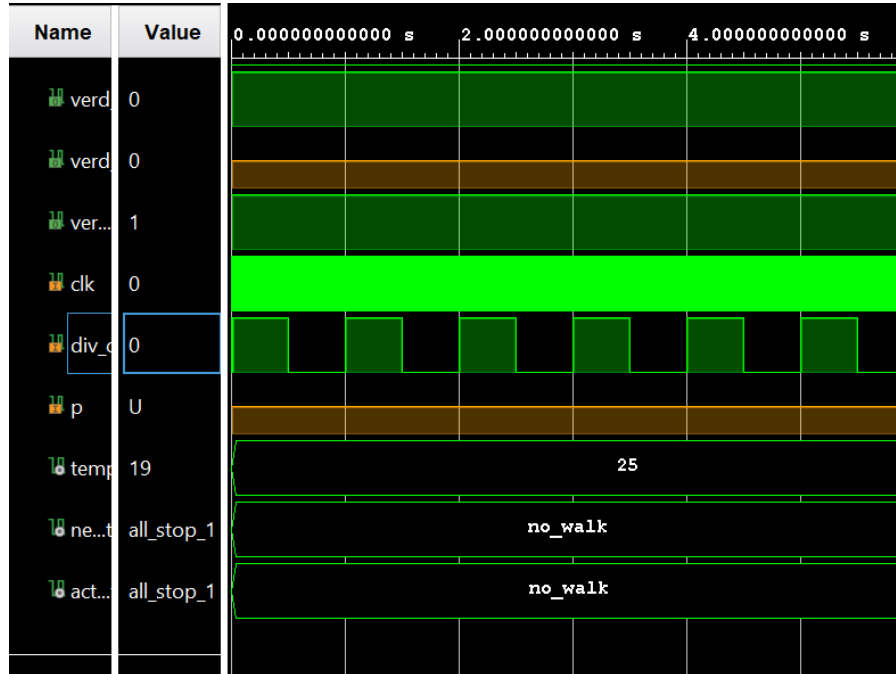
El descodificador de l'ànode selecciona el valor corresponent a cada instant de temps amb la variable *digit_act*.

```
142 | -- Selector d'ànode
143 | with digit_act select an <=
144 |         "1110" when 0,
145 |         "1101" when 1,
146 |         "1011" when 2,
147 |         "0111" when 3;
148 |
149 | end Behavioral;
```

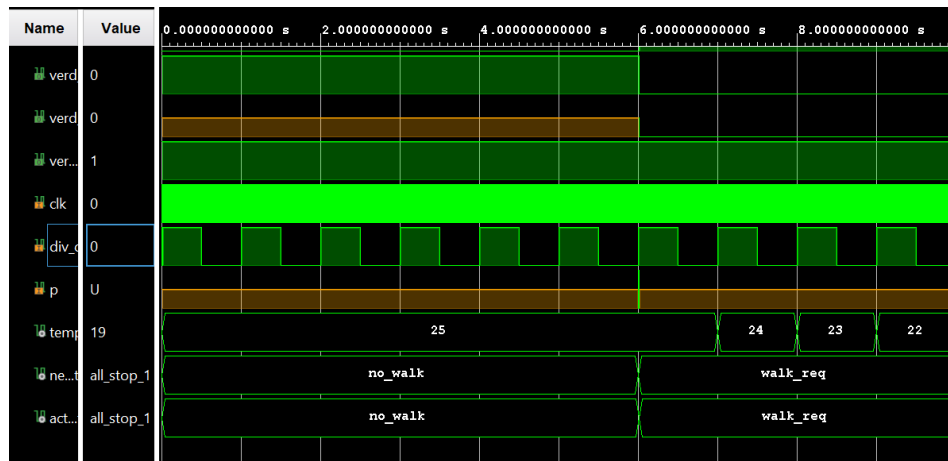
Finalment tanquem l'arquitectura.

3. SIMULACIÓ

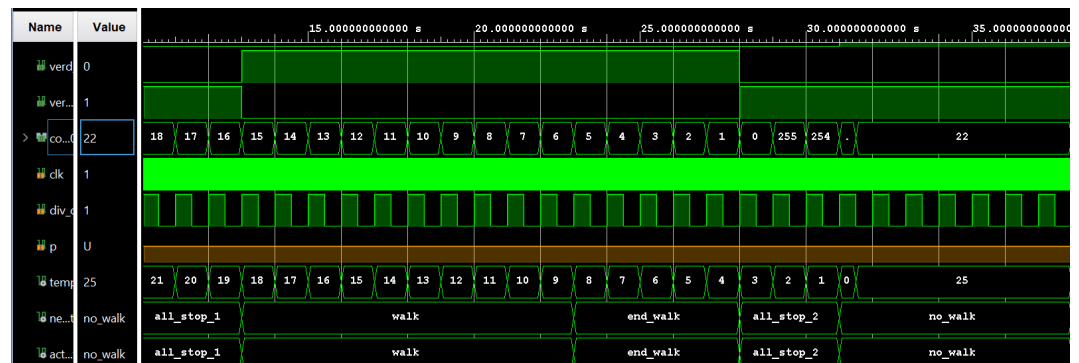
ESTAT INICIAL:



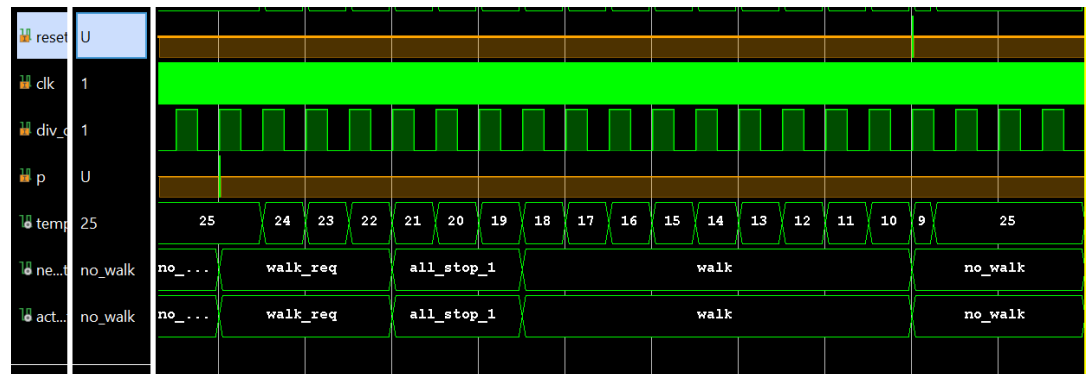
PREMEM EL BOTÓ DE PAS:



DEIXEM QUE PASSIN ELS 25 SEGONS:



FEM UN RESET AL MIG D'UN PROCÉS:





4. CONCLUSIONS

En aquesta pràctica hem implementat la lògica interna i la visualització d'un semàfor per a vehicles i peatons amb un botó asíncron. Això ha implicat desenvolupar una màquina d'estats que controli el funcionament del semàfor i un programa visualitzador que representi cada estat correctament en els visualitzador de la placa Basys 3. Hem pogut aprofundir en la comprensió de l'execució concurrent dels processos de VHDL en una FPGA i les conseqüències temporals que comporta.